ASSIGNMENT 4

Sammarth Kapse
9088922829
https://github.com/sammarth-kapse/repo759/tree/main/HW04

Task 1

I ran the supplied `nbody.py` locally without changing its source. It uses $N = 100$, $t_{\text{end}} = 10$, $\Delta t = 0.01$, and seed 17. The saved final particle/trail plot is submitted separately as `task1.png`.

Task 2

The serial C++ program completes the supplied skeleton. For each particle it computes

$$\mathbf{a}_i = G \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \epsilon^2)^{3/2}}.$$

The constants are $G = 1$, $\epsilon = 0.1$, $\Delta t = 0.01$, and board limits $[-4, 4]$. Velocities are converted to the center-of-mass frame. Each timestep applies a half kick, drifts and clamps positions, recomputes acceleration, and applies the second half kick. The timestep count follows the supplied C++ skeleton.

Task 3

OpenMP parallel loops distribute particles among threads. Every force-loop iteration owns one output acceleration and keeps its summation over other particles serial. The implicit barriers finish each stage before the next stage uses its results. `schedule(runtime)` and `OMP_SCHEDULE` select static, dynamic, or guided without changing the required command-line interface. Both C++ versions use seed 17; NumPy and C++ generators produce different initial samples, so their plots need not coincide.

Validation: On Euler, 60 force checks, 50 full-trajectory checks, and 10 invalid-input checks passed. The largest position difference from an independent Python reference was 4.44×10^{-16}. Both required compile commands worked without loading additional modules. Per-step CSV output is disabled in benchmark builds.

Task 4

All experiments used Euler node `euler01.engr.wisc.edu`, Slurm job 108475, and exactly eight requested CPUs. The workload was $N = 256$, $t_{\text{end}} = 100$, $\Delta t = 0.01$ (10000 timesteps). Each point is the median of 3 fresh-process runs. Timings include C++ initialization, the initial acceleration, and the simulation loop; CSV I/O and printing are excluded.

Policy	1 thread (ms)	8 threads (ms)	Best t	Best speedup
Static	2331.6	771.53	4	$3.141\times$
Dynamic	2393.7	1296.6	8	$1.8461\times$
Guided	2337.3	908.58	4	$2.7246\times$

Speedups use the same policy at one thread as the baseline.

The serial Task 2 baseline was 2311.7 ms. The fastest measured OpenMP configuration was static scheduling with 4 threads (742.31 ms). Static scheduling assigns fixed chunks with little dispatch overhead. Dynamic scheduling hands out chunks as threads become free; guided scheduling uses progressively smaller chunks. The force loop has almost equal work per particle, so frequent dynamic dispatch can add overhead without fixing much imbalance. Repeated parallel regions and barriers, and shared hardware, can also limit scaling. These are possible explanations; the measurements do not isolate each cost.

The three required linear–linear plots are included in `task4.pdf`, and are reproduced on the following page with identical axis limits.

Task 4: Scheduling plots





